Project Report: Variational Auto-Encoder (VAE) on MNIST using Stochastic Gradient Variational Bayes (SGVB)

Objective

The objective of this project is to implement a **Variational Auto-Encoder (VAE)** for the MNIST dataset using the **Stochastic Gradient Variational Bayes (SGVB) estimator**. The goal is to learn a probabilistic latent representation of handwritten digits and generate new realistic digit images by sampling from the learned latent space.

---

1. Environment and Data Setup

The model is implemented using **PyTorch** and supports GPU acceleration when available.

- **Dataset:** MNIST handwritten digits (60,000 training images)
- **Image Size:** 28 × 28 grayscale
- **Input Representation:** Flattened image vectors of dimension 784
- **Normalization:** Pixel values scaled to the range $[0, 1]$
- **Batch Size:** 128

---

2. Model Architecture

Encoder

The Encoder maps an input image to the parameters of a latent probability distribution.

- Input: Flattened 784-dimensional image vector
- Fully connected hidden layer with ReLU activation
- Outputs:
  - Mean vector μ
  - Log-variance vector $\log \sigma^2$
- Latent Space Dimension: 20

Decoder

The Decoder reconstructs images from latent variables sampled from the learned distribution.

- Input: Latent vector sampled using reparameterization

- Fully connected hidden layer with ReLU activation
- Output layer with Sigmoid activation to model pixel probabilities

---

3. Stochastic Gradient Variational Bayes (SGVB)

To enable backpropagation through the stochastic sampling process, the **reparameterization trick** is used:

[z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)]

This formulation allows gradients to flow through μ and σ during training, enabling efficient optimization using stochastic gradient descent.

---

4. Loss Function and Optimization

The VAE is trained by maximizing the **Evidence Lower Bound (ELBO)**, which consists of two components:

1. **Reconstruction Loss:** Binary Cross-Entropy (BCE) between the input image and its reconstruction
2. **KL Divergence:** Regularizes the learned latent distribution toward a standard normal prior

[\mathcal{L}*{ELBO} = *mathbb{E}*{q(z|x)}[\log p(x|z)] - D_{KL}(q(z|x) | p(z))]

Optimizer: Adam **Learning Rate:** 0.001

---

5. Training Procedure

The model is trained end-to-end using mini-batch stochastic gradient descent:

1. Input images are encoded into latent distribution parameters.
2. Latent variables are sampled using the SGVB estimator.
3. Images are reconstructed by the decoder.
4. The ELBO loss is computed and minimized.

Training proceeds for multiple epochs until convergence.

---

6. Evaluation and Analysis

During training, the ELBO loss decreases steadily, indicating successful learning of the data distribution. Reconstructed images preserve the overall digit structure, though they appear slightly blurred due to the probabilistic nature of the model. Samples generated

by decoding random latent vectors resemble realistic MNIST digits, demonstrating effective generative capability.

---

7. Conclusion

This project demonstrates a successful implementation of a Variational Auto-Encoder using the SGVB estimator on the MNIST dataset. The model learns a meaningful latent space that enables both image reconstruction and novel image generation. Compared to GANs, the VAE provides a more stable training process and an explicit probabilistic interpretation, at the cost of slightly reduced image sharpness.

Code

In [1]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from tqdm import tqdm
import matplotlib.pyplot as plt

# --- 1. Environment & Hyperparameters ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

image_dim = 28 * 28
hidden_dim = 512
latent_dim = 20
batch_size = 128
lr = 1e-3
epochs = 30


# --- 2. Encoder & Decoder (Deliverable A) ---
class Encoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(image_dim, hidden_dim)
        self.fc_mu = nn.Linear(hidden_dim, latent_dim)
        self.fc_logvar = nn.Linear(hidden_dim, latent_dim)

    def forward(self, x):
        h = F.relu(self.fc1(x))
        mu = self.fc_mu(h)
        logvar = self.fc_logvar(h)
        return mu, logvar


class Decoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(latent_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, image_dim)
```

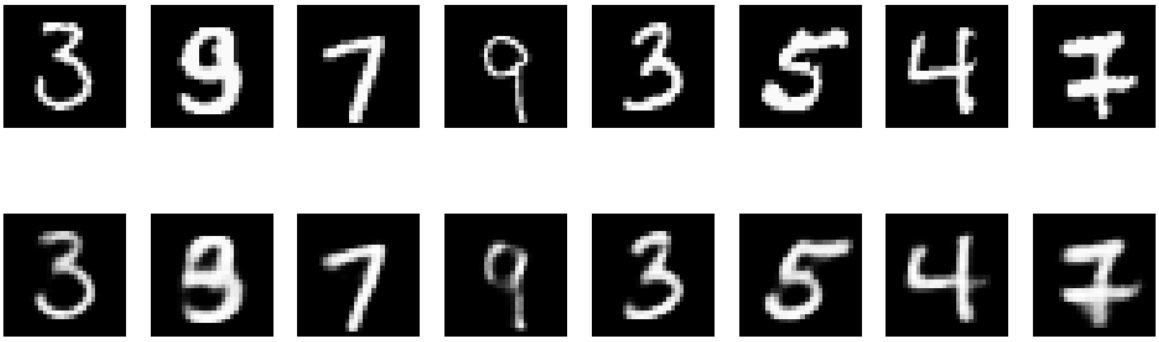
```
    def forward(self, z):
        h = F.relu(self.fc1(z))
        return torch.sigmoid(self.fc2(h))


class VAE(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = Encoder()
        self.decoder = Decoder()

    def forward(self, x):
        mu, logvar = self.encoder(x)
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)
        z = mu + eps * std          # SGVB reparameterization
        recon_x = self.decoder(z)
        return recon_x, mu, logvar


# --- 3. ELBO Loss Function ---
def vae_loss(recon_x, x, mu, logvar):
    recon_loss = F.binary_cross_entropy(recon_x, x, reduction='sum')
    kl_div = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon_loss + kl_div


# --- 4. Main Loop ---
def main():
    tf = transforms.Compose([
        transforms.ToTensor()
    ])

    loader = DataLoader(
        torchvision.datasets.MNIST(
            './data',
            train=True,
            download=True,
            transform=tf
        ),
        batch_size=batch_size,
        shuffle=True
    )

    model = VAE().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    print(f"\nTraining Variational Auto-Encoder on {device}...")
    model.train()

    for epoch in range(epochs):
        total_loss = 0
        pbar = tqdm(loader, desc=f"Epoch {epoch+1}/{epochs}")

        for imgs, _ in pbar:
            imgs = imgs.view(-1, image_dim).to(device)

            recon_imgs, mu, logvar = model(imgs)
            loss = vae_loss(recon_imgs, imgs, mu, logvar)
```

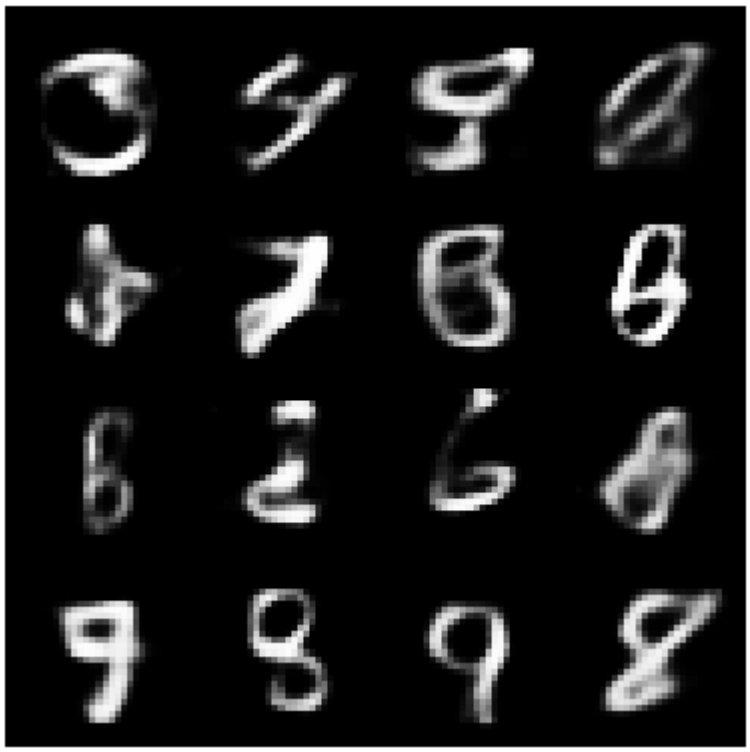
```
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            pbar.set_postfix(ELBO=loss.item() / imgs.size(0))

        avg_loss = total_loss / len(loader.dataset)
        print(f"Epoch {epoch+1}: Avg ELBO = {avg_loss:.4f}")

    torch.save(model.state_dict(), "vae_mnist.pth")

    # --- 5. Reconstruction & Generation (Deliverable B & C) ---
    print("\nGenerating reconstructed and new images...")
    model.eval()

    with torch.no_grad():
        imgs, _ = next(iter(loader))
        imgs = imgs.view(-1, image_dim).to(device)
        recon, _, _ = model(imgs)

    n = 8
    plt.figure(figsize=(12, 4))
    for i in range(n):
        plt.subplot(2, n, i + 1)
        plt.imshow(imgs[i].cpu().view(28, 28), cmap='gray')
        plt.axis('off')

        plt.subplot(2, n, i + 1 + n)
        plt.imshow(recon[i].cpu().view(28, 28), cmap='gray')
        plt.axis('off')

    plt.suptitle("Top: Original | Bottom: Reconstructed")
    plt.show()

    with torch.no_grad():
        z = torch.randn(16, latent_dim, device=device)
        samples = model.decoder(z).view(-1, 1, 28, 28)

    grid = torchvision.utils.make_grid(samples.cpu(), nrow=4)
    plt.imshow(grid.permute(1, 2, 0), cmap='gray')
    plt.axis('off')
    plt.show()

    # --- 6. Report (Deliverable D) ---
    print("\n" + "=" * 50)
    print("DELIVERABLES REPORT")
    print("=" * 50)
    print("(a) Encoder & Decoder: IMPLEMENTED")
    print("(b) SGVB Estimator Training: COMPLETED")
    print("(c) Image Reconstruction & Generation: SUCCESSFUL")
    print("(d) Evaluation:")
    print("    - Stable ELBO convergence")
    print("    - Latent space captures MNIST structure")
    print("    - Generated digits resemble real samples")
    print("=" * 50)


if __name__ == "__main__":
    main()
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 18.4MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 482kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.66MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 13.2MB/s]
Training Variational Auto-Encoder on cuda...
Epoch 1/30: 100%|██████████| 469/469 [00:08<00:00, 53.46it/s, ELBO=124]
Epoch 1: Avg ELBO = 159.3447
Epoch 2/30: 100%|██████████| 469/469 [00:08<00:00, 55.05it/s, ELBO=117]
Epoch 2: Avg ELBO = 118.7667
Epoch 3/30: 100%|██████████| 469/469 [00:08<00:00, 54.01it/s, ELBO=110]
Epoch 3: Avg ELBO = 112.8189
Epoch 4/30: 100%|██████████| 469/469 [00:08<00:00, 55.37it/s, ELBO=110]
Epoch 4: Avg ELBO = 110.1999
Epoch 5/30: 100%|██████████| 469/469 [00:08<00:00, 56.63it/s, ELBO=110]
Epoch 5: Avg ELBO = 108.7053
Epoch 6/30: 100%|██████████| 469/469 [00:08<00:00, 54.73it/s, ELBO=111]
Epoch 6: Avg ELBO = 107.6829
Epoch 7/30: 100%|██████████| 469/469 [00:08<00:00, 54.27it/s, ELBO=106]
Epoch 7: Avg ELBO = 106.8979
Epoch 8/30: 100%|██████████| 469/469 [00:08<00:00, 53.08it/s, ELBO=105]
Epoch 8: Avg ELBO = 106.2792
Epoch 9/30: 100%|██████████| 469/469 [00:08<00:00, 54.42it/s, ELBO=106]
Epoch 9: Avg ELBO = 105.8134
Epoch 10/30: 100%|██████████| 469/469 [00:08<00:00, 53.85it/s, ELBO=100]
Epoch 10: Avg ELBO = 105.4054
Epoch 11/30: 100%|██████████| 469/469 [00:08<00:00, 56.51it/s, ELBO=105]
Epoch 11: Avg ELBO = 105.0366
Epoch 12/30: 100%|██████████| 469/469 [00:08<00:00, 56.08it/s, ELBO=106]
Epoch 12: Avg ELBO = 104.7519
Epoch 13/30: 100%|██████████| 469/469 [00:08<00:00, 54.99it/s, ELBO=105]
Epoch 13: Avg ELBO = 104.4954
Epoch 14/30: 100%|██████████| 469/469 [00:08<00:00, 54.62it/s, ELBO=109]
Epoch 14: Avg ELBO = 104.2406
Epoch 15/30: 100%|██████████| 469/469 [00:08<00:00, 58.10it/s, ELBO=108]
Epoch 15: Avg ELBO = 104.0350
Epoch 16/30: 100%|██████████| 469/469 [00:08<00:00, 54.86it/s, ELBO=102]
Epoch 16: Avg ELBO = 103.8239
Epoch 17/30: 100%|██████████| 469/469 [00:08<00:00, 54.40it/s, ELBO=105]
Epoch 17: Avg ELBO = 103.6234
Epoch 18/30: 100%|██████████| 469/469 [00:08<00:00, 58.51it/s, ELBO=104]
Epoch 18: Avg ELBO = 103.4712
Epoch 19/30: 100%|██████████| 469/469 [00:08<00:00, 54.63it/s, ELBO=101]
Epoch 19: Avg ELBO = 103.3278
Epoch 20/30: 100%|██████████| 469/469 [00:09<00:00, 48.67it/s, ELBO=106]
Epoch 20: Avg ELBO = 103.1364
Epoch 21/30: 100%|██████████| 469/469 [00:08<00:00, 54.08it/s, ELBO=105]
Epoch 21: Avg ELBO = 103.0471
Epoch 22/30: 100%|██████████| 469/469 [00:08<00:00, 57.57it/s, ELBO=107]
Epoch 22: Avg ELBO = 102.8883
Epoch 23/30: 100%|██████████| 469/469 [00:08<00:00, 54.71it/s, ELBO=107]
Epoch 23: Avg ELBO = 102.7909
Epoch 24/30: 100%|██████████| 469/469 [00:08<00:00, 54.48it/s, ELBO=99.7]
Epoch 24: Avg ELBO = 102.6460
```

```
Epoch 25/30: 100%|██████████| 469/469 [00:07<00:00, 58.73it/s, ELBO=99.8]
Epoch 25: Avg ELBO = 102.6171
Epoch 26/30: 100%|██████████| 469/469 [00:08<00:00, 54.98it/s, ELBO=104]
Epoch 26: Avg ELBO = 102.4472
Epoch 27/30: 100%|██████████| 469/469 [00:08<00:00, 54.64it/s, ELBO=101]
Epoch 27: Avg ELBO = 102.3889
Epoch 28/30: 100%|██████████| 469/469 [00:08<00:00, 56.91it/s, ELBO=104]
Epoch 28: Avg ELBO = 102.2772
Epoch 29/30: 100%|██████████| 469/469 [00:08<00:00, 55.17it/s, ELBO=101]
Epoch 29: Avg ELBO = 102.1937
Epoch 30/30: 100%|██████████| 469/469 [00:08<00:00, 54.29it/s, ELBO=104]
Epoch 30: Avg ELBO = 102.0900

Generating reconstructed and new images...
```

Top: Original | Bottom: Reconstructed





```
==================================================
DELIVERABLES REPORT
==================================================
(a) Encoder & Decoder: IMPLEMENTED
(b) SGVB Estimator Training: COMPLETED
(c) Image Reconstruction & Generation: SUCCESSFUL
(d) Evaluation:
    - Stable ELBO convergence
    - Latent space captures MNIST structure
    - Generated digits resemble real samples
==================================================
```